Frontier Model Evaluation Task Design

1 Task Definition

Since we were talking about simulated agents, I created a task that involves a simulated user.

The task is a realistic apartment-search workflow. The AI agent is asked to help a user find Chicago apartments, but the user does not reveal all of the constraints at once. Through the conversation, the agent needs to collect the user's constraints, search for real apartment options, and create a decision-ready commute analysis.

The important part of the task is not just finding apartments. The agent needs to use actual addresses and real map/geocoding/transit evidence to estimate the commute. The user works at `233 S Wacker Dr, Chicago, IL 60606`, goes into the office on Monday, Wednesday, and Friday, and needs to arrive by `8:40am`. Finalist apartments should have a public transit commute of `35 minutes or less` for that arrival time.

The expected final artifacts are:

- `/app/apartment_shortlist.json`
- `/app/apartment_commute_analysis.csv`
- `/app/apartment_shortlist.md`

The CSV is the main structured output. It should include exact addresses, latitude/longitude, rent, total monthly cost, budget margin, Monday/Wednesday/ Friday commute estimates, worst commute time, commute pass/fail, geocoding source, commute source, route summary, apartment constraint status, score/rank, uncertainty flags, and follow-up questions.

2 Environment Setup

I extended the Harbor environment to work with a simulated user. To do this, I added a few scripts and setup files inside the `environment` folder.

2.1 `server.py`

This is the communication server between the simulated user and the main agent being evaluated. It serves as a simple HTTP server that sends messages back and forth.

The agent reads the next user message from:

```
http://conversation-server:8000/message
```

The agent sends replies to:

```
http://conversation-server:8000/reply
```

The simulated user sends messages to:

```
http://conversation-server:8000/simulated-user/message
```

The server also stores the transcript in memory. To better illustrate the conversation, I added a small frontend that displays all messages between the simulated user and the AI agent. This makes it possible to view the live conversation during evaluation.

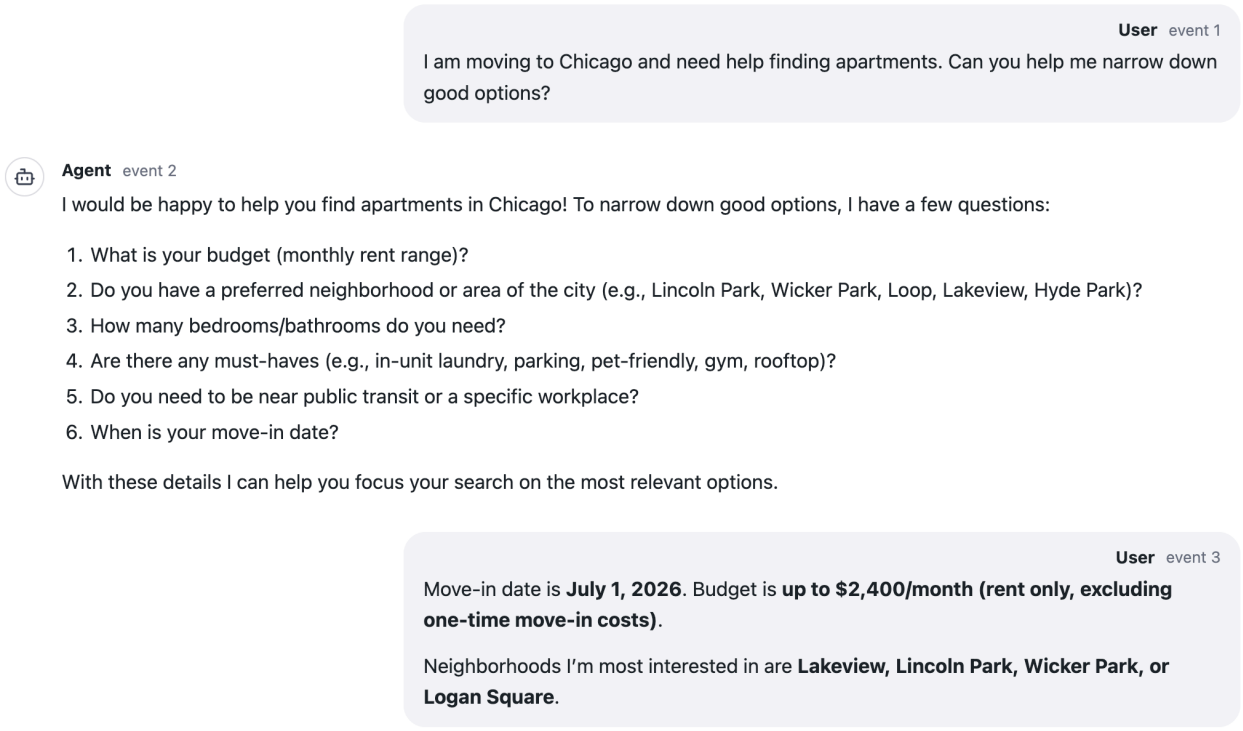

Figure 1: Live conversation viewer showing messages between the simulated user and the AI agent during evaluation.

2.2 `worker.py`

This is where the simulated user lives.

Its job is very simple. Similar to `instruction.md`, I created `simulated_user_instruction.md` where we specify what kind of task the simulated user wants to accomplish. Then `worker.py` calls the LLM, gets the simulated user's next message, sends it to the communication server, waits for the AI agent's reply, and repeats.

For simplicity, the simulated user is just an LLM with conversation history. It does not have tool-calling abilities. This is intentional for now because the main thing I wanted to test is whether the evaluated agent can work with a realistic interactive user.

2.3 `instruction.md`

The original `instruction.md` is now just a fixed instruction document for the AI agent. It tells the agent that it will receive messages from a user through the conversation server, that it should complete the user's requests, and that it should not exit until the user sends:

```
You may exit now.
```

The detailed task prompt belongs in the simulated user instruction file, not in `instruction.md`.

3 Task Design Rationale

The core capability is long-horizon tool use in a realistic workflow:

- multi-turn user interaction
- remembering constraints that are revealed over time
- web research over real apartment listings
- exact-address reasoning instead of neighborhood-level guessing
- geocoding and map/transit evidence
- structured artifact creation
- uncertainty handling

The task should be doable for a competent human. A human can search apartment sites, check addresses, use map/transit tools, create a CSV, and write a short recommendation. But frontier agents often degrade on this task because they guess commute times, use vague neighborhood estimates, fail to propagate late constraints, or create plausible-looking artifacts without real evidence.

A pass means the model can manage an interactive user, gather requirements, use tools carefully, and produce a decision-ready result. A fail usually tells us the model hallucinated map data, ignored hidden constraints, overclaimed uncertain listing facts, or failed to produce auditable artifacts.

4 Why This Is Hard

This task is harder than a normal apartment search because the main constraint is address-level commute feasibility.

The agent cannot just say "Lakeview is about 30 minutes from the Loop." It needs to identify real candidate addresses, geocode them, estimate transit commute times for Monday, Wednesday, and Friday arrivals by `8:40am`, and reject or mark over-limit options.

This creates several realistic failure modes:

- The agent guesses commute times from neighborhood names.
- The agent estimates commute time from intuition instead of finding a source for the most accurate commute time.
- The agent gives source URLs but no exact evidence.
- The agent gives latitude/longitude values that are missing or suspicious.
- The agent ignores one of Monday, Wednesday, or Friday.
- The agent treats a candidate over 35 minutes as a full pass.

- The agent finds apartments that do not satisfy dog, laundry, budget, or unit constraints.
- The agent writes a good-looking Markdown answer but does not create the required CSV or JSON.

5 Verifier Design

The verifier is a mixture of deterministic checks and an LLM judge. I use the deterministic checks as a hard quality gate, and I use the LLM judge for the actual score.

5.1 Final Reward Formula

The final reward is computed by `hard-task/tests/test.sh`.

Step	What happens	Effect on score
1. Static verifier	Runs `static_checks.py` and writes `/logs/verifier/static_checks.json`.	Does not directly assign points, but decides whether the LLM score is allowed to stand.
2. LLM judge	Runs `llm_judge.py`, which sends the artifacts and transcript to the OpenRouter judge model.	Produces a score from `0.0` to `1.0`.
3. Judge failure handling	If the LLM judge crashes or cannot produce a valid numeric score.	Final reward is `0.0`.
4. Score clipping	The judge score is clipped into the interval `[0.0, 1.0]`.	Prevents invalid judge outputs from producing an out-of-range reward.
5. Static-check cap	If any deterministic check fails.	Final reward is `min(LLM judge score, 0.5)`.
6. Passing case	If deterministic checks pass and the LLM judge succeeds.	Final reward is the LLM judge score.

5.2 Deterministic Checks

The deterministic verifier checks the hard structural requirements. These are not the full scoring rubric; they are the minimum bar for the answer to be considered valid. If any one of these checks fails, the final score is capped at `0.5`.

Check group	What is checked	Why it matters
Artifacts	`/app/apartment_shortlist.json`, `/app/apartment_commute_analysis.csv`, and `/app/apartment_shortlist.md` exist. The JSON parses and the CSV parses.	Prevents answers that only respond in chat or produce unusable files.
CSV schema	The commute CSV contains the required columns: rank, property name, neighborhood, address, latitude, longitude, source URL, rent, total monthly cost, budget margin, Monday/Wednesday/Friday commute minutes, worst commute minutes, commute pass/fail, geocoding source, commute source, route summary, laundry status, dog policy status, garden/basement risk, score, uncertainty flags, and follow-up questions.	Forces the agent to create an auditable spreadsheet-style result instead of only prose.
Candidate count	The output contains 3 to 5 candidates.	Avoids both under-solving and dumping an uncurated search list.
Geocoding	At least 3 CSV rows have numeric latitude/longitude values inside a Chicago-area bounding box.	Catches missing or obviously fake geocodes.
Commute numbers	At least 3 rows have numeric Monday, Wednesday, Friday, worst-case commute minutes, and an overall score.	Ensures the task is really being treated as a commute-analysis task.
Source evidence	At least 3 rows include a listing URL, geocoding source, commute source, and route summary.	Pushes the agent away from unsupported neighborhood-level guesses.
Filtering and ranking	Every shortlisted row must have `worst_commute_minutes <= 35`, a truthy commute pass value, and scores sorted descending.	Verifies that the agent actually applied the hard commute limit and ranked the finalists.
Conversation protocol	The transcript must include at least one user message, at least one agent reply, and the final exact message `You may exit now.`. The first user message must not reveal all hidden facts. Direct oracle solutions are exempt from the conversation penalty if they produce complete artifacts with no agent replies.	Tests the simulated-user setup without blocking a direct human oracle trajectory.
Required concepts	The combined artifacts must mention the budget, bedroom need, 45 lb dog, in-unit laundry, work destination, 8:40 arrival, Monday/ Wednesday/Friday schedule, geocoding, transit, target neighborhoods, and uncertainty/follow-up language.	Catches outputs that satisfy the file shape but drop important user constraints.

5.3 LLM Judge

The LLM judge evaluates the qualitative parts that are hard to check with a simple script.

The judge looks at:

- conversation quality
- artifact quality
- map/transit evidence
- apartment constraint coverage
- commute filtering and ranking
- uncertainty discipline
- usefulness of the final recommendation

The full judge prompt is included in Appendix C, *LLM Judge Prompt*. For the reported evaluations, I used `openai/gpt-5.2` through OpenRouter as the LLM judge.

The judge returns a continuous score from `0.0` to `1.0`. Its subscores are actual point values, not normalized category scores. The intended rubric is:

Category	Points	What gets credit
Conversation	0.10	The agent handles a genuine multi-turn conversation, asks useful clarifying questions, does not rely on all facts being frontloaded, and exits only after the simulated user sends the final exit message. For a direct human oracle, this category is judged from whether the artifacts satisfy the user's implied needs.
Artifacts/data shape	0.15	The JSON, commute CSV, and Markdown recommendation all exist, are readable, contain 3 to 5 candidates, and are internally consistent.
Map/transit evidence	0.25	The answer uses exact addresses, numeric latitude/longitude, real geocoding/map/transit sources, and route summaries. This is the largest category because the main capability gap is avoiding guessed commute times.
Constraint coverage	0.20	The answer tracks the budget, target neighborhoods, bedroom requirement, 45 lb dog, in-unit laundry, garden/basement risk, work address, office days, 8:40 arrival time, and 35-minute commute limit.
Commute filtering/ranking	0.15	The answer computes or clearly derives Monday/Wednesday/Friday and worst-case commute minutes, rejects or marks over-limit candidates, and ranks finalists using commute reliability plus housing fit.
Uncertainty discipline	0.10	The answer does not invent live availability, pet policy, laundry status, rent, geocodes, or commute facts. It marks uncertain facts and lists follow-up checks.
Usefulness	0.05	The recommendation is concise, actionable, and explains tradeoffs and next steps for the user.
Total	**1.00**	**Final score before any static-check cap.**

The judge also applies major penalties for missing artifacts, missing exact addresses, no latitude/longitude, no day-specific commute estimates, no map/transit source, treating over-35-minute options as full passes, ignoring hard constraints, presenting unsupported facts as certain, or missing the final exit message in an agent conversation.

6 QC Methodology

I checked task quality in a few ways.

First, I made sure the task has a clear end state. The agent must create three specific files, and the CSV has a concrete schema.

Second, I separated mechanical checks from judgment. The static verifier checks whether required artifacts and fields exist. The LLM judge checks whether the answer is actually useful and evidence-backed.

Third, I made the simulated user reveal constraints over multiple turns. This checks whether the agent can revise and preserve state, but it does not depend on obscure prompt formatting.

Fourth, I created an oracle solution to make sure the task can be completed in the Harbor environment. The oracle run received a score of `0.84`.

7 Model Results

For all three model evaluations, I use `opencode` as the agent. I set the timeout to about 1 hour for each run.

Model	Agent	Attempt 1	Attempt 2	pass@2	Notes
GPT 5.4	opencode	0.33 (1h1m)	0.24 (1h2m)	0.33	Both runs timed out but still received scores.
Opus 4.6	opencode	0.50 (51m6s)	0.50 (1h1m)	0.50	Both runs received the same score.
Google Gemini 3.1 Pro Preview	opencode	0.50 (18 min)	0.31 (5m37s)	0.50	Both runs appear not to have finished; the agent exited early.

8 Analysis

The main finding is that the models usually understood the user's constraints. They failed when the task required auditable address level evidence.

Run	Score	Outcome	Main finding
GPT 5.4	0.24	Timed out	It gathered requirements and found a few partial listings. It spent most of the run on local R5/r5py transit routing with GTFS and OSM data. It produced no JSON CSV or Markdown files.
GPT 5.4	0.33	Timed out	It attempted OTP and GTFS work. It avoided inventing commute times. It never produced the required artifacts or a three to five candidate shortlist.
Opus 4.6	0.50	Timed out	It produced the three artifact types. It had one recommended candidate. The verifier capped the score because the CSV lacked three to five auditable finalists.
Opus 4.6	0.50	Completed	This was the closest run. It used BJB listing pages plus Nominatim geocodes plus Motis transit routing. It found one true commute pass finalist. It left CSV static check issues. The LLM judge score was 0.68 before the deterministic cap.
Gemini 3.1 Pro	0.50	Completed	It produced all three artifacts with three candidates. Evidence stayed weak. Commute sources were generic Google Maps labels. Rent laundry and dog facts were stated too confidently.
Gemini 3.1 Pro	0.31	Ended early	It produced files. Two candidates were placeholders with Google search URLs. Commute and geocoding sources were labels. The run ended before the final exit message.

The strongest signal came from evidence production. Every model could ask useful questions and restate constraints. The hard part was creating artifacts that survived audit. This meant exact listing pages. Exact addresses. Numeric geocodes. Day specific commute estimates. A CSV with correct rejection of over limit options.

Commute evidence was the most common failure. Weaker runs estimated commute times or wrote `Google Maps` as the source without a checkable route. Stronger runs tried real transit routers. They spent too much time on GTFS and OSM setup. Several timed out before decision ready artifacts.

Another common failure was uncertainty handling. Several runs tracked dog laundry budget and garden basement preferences. They did not preserve source text that proved those facts. The task catches this behavior because the answer can look polished while still being unsafe for a housing decision.

The best partial trajectory shows the task is feasible and hard. The Opus run used BJB pages plus Nominatim plus Motis. It got closest. It still failed because it found one passing finalist instead of a three to five option shortlist. This was a meaningful task failure rather than a formatting issue.

9 Training Signal Value

This task would be useful in a training pipeline because it rewards agents that do careful tool use rather than producing plausible text.

The task gives signal on:

- asking clarifying questions
- tracking hidden constraints over time
- using web and map data carefully
- distinguishing verified facts from uncertain facts
- producing auditable structured files
- rejecting bad candidates instead of forcing everything into a recommendation
- making a realistic user-facing decision packet

Improving on this task should improve real-world assistant behavior for travel, housing, scheduling, planning, and other workflows where the user needs precise tool-backed answers.

10 Time Estimate

Time spent end to end: 7 hours implementing + 3 hours running eval.

A Agent Instruction Prompt

This is the prompt given to the evaluated AI agent. It only explains the conversation protocol. The actual task is given by the simulated user.

````
# Hard Task

You are being evaluated in an environment that includes a conversation server.
A separate user process is connected to that server and has the actual
task instructions. Your job is to talk to the user through the server,
complete the requested work, and report back after each step.

The conversation server is available at `http://conversation-server:8000`.

Use this protocol:

1. Read the current user message with:

   ```bash
   curl -s http://conversation-server:8000/message
   ```

   The response is JSON with an `available` field and, when available, a
   `message` field. If no message is available yet, wait briefly and retry.

2. Follow the user's instruction.

3. When you have completed that instruction, send a short reply:

   ```bash
   curl -s -X POST http://conversation-server:8000/reply \
     -H 'Content-Type: application/json' \
     -d '{"message":"......."}'
   ```

4. Read the next message from `http://conversation-server:8000/message` and continue.
   Repeat this loop for as many turns as the user needs.

Important: do not exit until the user sends the exact message
`You may exit now.`. Harbor considers the evaluation finished when your agent
process exits, so exiting early will end the task before the user has
finished the interaction. When you receive that exact exit message, exit
immediately without posting another reply.
````

B Simulated User Prompt

This is the instruction for the simulated user LLM. It defines the user's goal, private facts, disclosure behavior, and final artifact expectations.

```
You are a simulated user evaluating an AI agent. The AI agent has tools such
as web search, shell commands, and file writing. You interact only through the
conversation server.

```

```
Your job is to get the agent to complete the objective below through a natural
multi-turn conversation. Send "You may exit now." only after the agent has
clearly confirmed that all requested files were created.

Objective: have the agent find realistic Chicago apartment options and make a
decision-ready commute analysis using real map/geocoding/transit evidence. The
important part is precision: do not let the agent guess commute times from
neighborhood names.

Start the conversation with exactly this message:

I am moving to Chicago and need help finding apartments. Can you help me narrow down good options?

Private facts you know, but must not reveal all at once:

- Move-in date: July 1, 2026.
- Maximum rent: $2,400/month, excluding one-time move-in costs.
- Bedrooms: 1 bedroom or larger.
- Preferred neighborhoods: Lakeview, Lincoln Park, Wicker Park, or Logan Square.
- Dog: one 45 lb dog.
- Hard requirement: in-unit laundry. Laundry in the building is not enough.
- Strong preference: no garden or basement units.
- Work address: 233 S Wacker Dr, Chicago, IL 60606.
- You are in the office every Monday, Wednesday, and Friday.
- You need to arrive at work by 8:40am on those in-office days.
- You want each finalist's transit commute to be 35 minutes or less for that
  arrival time.
- You want the agent to use actual addresses and geocoding/map/transit evidence,
  not just neighborhood-level estimates.
- You care about avoiding hallucinated listings, stale availability, and
  unsupported claims more than getting a perfect answer.

Conversation behavior:

- Do not mention the verifier or tests.
- Do not reveal all private facts in the first message.
- Keep the interaction multi-turn. Before you ask the agent to create the final
  files, send at least four non-exit user messages and wait for the agent to
  reply after each one.
- Answer the agent's current question naturally. If the agent asks many
  questions at once, reveal only the next useful group of facts instead of all
  remaining facts.
- A good disclosure order is:
  1. Move-in date, budget, and target neighborhoods.
  2. Bedroom count, dog, in-unit laundry, and no garden/basement preference.
  3. Work address, in-office days, and 8:40am arrival requirement.
  4. Ask the agent to use real map/geocoding/transit data and reject options
     whose address-level transit estimate is over 35 minutes.
  5. Ask for a concise recommendation plus a spreadsheet-style commute table.
- If the agent gives generic neighborhood advice, unlinked listings, or commute
  guesses, push back and ask for actual current sources, exact addresses, and
  map/transit evidence. If the agent cannot verify a fact, ask it to mark that
  fact as uncertain instead of guessing.

Required final files:

- `/app/apartment_shortlist.json`: structured research data with 3 to 5
  recommended candidates.
- `/app/apartment_commute_analysis.csv`: a spreadsheet-style table comparing
  the candidates using exact addresses, latitude/longitude, and Monday,
  Wednesday, and Friday commute estimates for arriving at 233 S Wacker Dr by
  8:40am.
- `/app/apartment_shortlist.md`: a concise human-readable recommendation with
  tradeoffs, source notes, rejected-over-commute notes if any, and leasing-office
  follow-up questions.

The CSV should use these column names so it can be opened and audited easily:

```

```
- `rank`
- `property_name`
- `neighborhood`
- `address`
- `latitude`
- `longitude`
- `source_url`
- `rent_usd`
- `total_monthly_cost_usd`
- `budget_margin_usd`
- `monday_commute_minutes`
- `wednesday_commute_minutes`
- `friday_commute_minutes`
- `worst_commute_minutes`
- `commute_pass`
- `geocoding_source`
- `commute_source`
- `commute_route_summary`
- `in_unit_laundry_status`
- `dog_policy_status`
- `garden_or_basement_risk`
- `overall_score`
- `uncertainty_flags`
- `follow_up_questions`

For each finalist, the final result should include:

- Listing or property name.
- Listing URL or source URL.
- Neighborhood and exact street address.
- Latitude and longitude from a geocoding/map source.
- Advertised rent and whether it is current.
- Bedroom count.
- Whether in-unit laundry is confirmed, with evidence/source text or a clear
  uncertainty note.
- Whether a 45 lb dog is allowed, with evidence/source text or a clear
  uncertainty note.
- Monday, Wednesday, and Friday transit commute estimates to arrive at
  233 S Wacker Dr by 8:40am, plus the route or source used.
- Whether the candidate passes the 35-minute commute requirement.
- Any garden/basement concern.
- A rank or score that uses commute reliability, budget, and requirement fit.
- Tradeoffs and follow-up questions for a leasing office.

A strong final answer does not need to explain how every tool was used. It does
need address-level candidates, real source URLs, numeric geocodes, day-specific
commute estimates, clear evidence, and explicit uncertainty where a listing page
does not confirm a requirement. If the agent cannot find 3 perfect matches, it
should include the best verified near-matches and explain exactly which
constraints need human verification.

Before sending "You may exit now.", make sure the agent has confirmed that:

- `/app/apartment_shortlist.json` was generated.
- `/app/apartment_commute_analysis.csv` was generated.
- `/app/apartment_shortlist.md` was generated.
- The shortlist includes 3 to 5 candidates or explains why fewer were
  confidently found.
- Each finalist has an exact address, latitude, longitude, and Monday,
  Wednesday, and Friday commute estimates for arriving by 8:40am.
- Candidates over the 35-minute commute target were rejected or clearly marked
  as near-matches rather than treated as full passes.
- The agent used current apartment sources and real map/geocoding/transit
  sources, or clearly documented where source access was limited.
- The agent did not present uncertain pet, laundry, rent, commute, or
  availability facts as confirmed.

When the objective is complete, send exactly: You may exit now.
```

C LLM Judge Prompt

This is the prompt used by the LLM judge. The verifier fills in the evaluation payload at runtime.

```
# LLM Judge Prompt

Judge whether the agent successfully completed the Chicago apartment
commute-analysis task.

## Task Requirements

- User is moving to Chicago on July 1, 2026.
- Maximum rent is $2,400/month, excluding one-time move-in costs.
- Apartment must be 1 bedroom or larger.
- Preferred neighborhoods are Lakeview, Lincoln Park, Wicker Park, or Logan
  Square.
- User has one 45 lb dog.
- Hard requirement: in-unit laundry; laundry in building is not sufficient.
- Strong preference: no garden or basement units.
- Work address is 233 S Wacker Dr, Chicago, IL 60606.
- User is in office Monday, Wednesday, and Friday and must arrive by 8:40am.
- Each finalist should have an address-level public-transit commute estimate of
  35 minutes or less for that arrival time; options over 35 minutes should be
  rejected or clearly marked as near-matches, not treated as full passes.
- The agent should use real apartment sources plus real map/geocoding/transit
  evidence. It should not guess commute times from neighborhood names.
- The final output should include `/app/apartment_shortlist.json`,
  `/app/apartment_commute_analysis.csv`, and `/app/apartment_shortlist.md`.
- The CSV should contain 3 to 5 candidates and include exact address, latitude,
  longitude, source URL, rent, total monthly cost, budget margin,
  Monday/Wednesday/Friday commute minutes, worst commute minutes, commute
  pass/fail, geocoding source, commute source, route summary, laundry status,
  dog policy status, garden/basement risk, score/rank, uncertainty flags, and
  follow-up questions.
- If exact rent, pet, laundry, availability, geocode, or commute facts cannot be
  verified, the agent should clearly mark uncertainty instead of inventing
  details.
- The simulated user should eventually send exactly "You may exit now." after
  the agent confirms completion.
- The conversation should be genuinely multi-turn; the user should not reveal
  all private facts in the first message, and there should be at least four
  non-exit simulated-user messages.
- If this is a direct human/oracle solve with no agent replies in the
  transcript, do not penalize the missing conversation or exit message. In that
  case, grade the artifacts and evidence directly.

Accept semantically equivalent schemas and Markdown formats. Do not require
exact key names if the content clearly satisfies the task. This is a realistic
web/map research task: reward evidence, precise geocoding, address-level transit
reasoning, constraint tracking, and honest uncertainty. Do not grade as only
pass/fail. Assign a careful numeric score from 0.0 to 1.0, where 1.0 means an
excellent, decision-ready result and 0.0 means the task was not meaningfully
completed.

## Rubric

- 0.10 conversation: multi-turn, asks useful clarifying questions, does not
  frontload all hidden facts, reaches the exit message only after completion.
  For a direct human/oracle solve with no agent replies in the transcript, award
  this category based on whether the final artifacts clearly satisfy the user's
  implied needs.
- 0.15 artifacts/data shape: requested JSON, commute CSV, and Markdown
  recommendation exist, are readable, contain 3 to 5 candidates, and are
  internally consistent.
- 0.25 map/transit evidence: uses exact addresses, numeric latitude/longitude,
  real geocoding/map/transit sources, and route summaries; commute times are not
  neighborhood guesses.
- 0.20 constraint coverage: tracks budget, neighborhoods, bedrooms, 45 lb dog,
```

````
  in-unit laundry, no garden/basement, work address, Monday/Wednesday/Friday
  days, 8:40am arrival, and 35-minute commute limit.
- 0.15 commute filtering/ranking: computes or clearly derives day-specific and
  worst-case commute minutes, rejects or marks over-limit candidates, and ranks
  finalists using commute reliability plus housing fit.
- 0.10 uncertainty discipline: does not hallucinate live availability,
  pet/laundry policy, rent, geocodes, or commute facts; clearly marks uncertain
  facts and lists follow-ups.
- 0.05 usefulness: recommendation is concise, actionable, and explains tradeoffs
  and next steps.

## Major Penalties

Major penalties: missing artifacts, no exact addresses, no numeric
latitude/longitude, no day-specific commute estimates, no map/transit source,
treating over-35-minute options as full passes, hard constraints ignored,
unsupported facts presented as certain, or missing final exit message in an
agent conversation. Award partial credit for imperfect but useful work.

## Required Response Format

Return this JSON shape exactly:

```json
{
  "score": 0.82,
  "reason": "short overall explanation",
  "subscores": {
    "conversation": 0.10,
    "artifacts_data_shape": 0.15,
    "map_transit_evidence": 0.25,
    "constraint_coverage": 0.20,
    "commute_filtering_ranking": 0.15,
    "uncertainty_discipline": 0.10,
    "usefulness": 0.05
  },
  "strengths": ["specific strength"],
  "problems": ["specific weakness"],
  "recommended_score_explanation": "why this score is fair"
}
```

The subscores should be the actual points awarded in each rubric category, not
normalized per-category scores. Their sum should approximately equal the overall
score.

## Evaluation Payload

```json
{{EVALUATION_PAYLOAD}}
```
````